KurveRSC 0.2.0: Task-Conditioned Relational Signal Compression

Schema-Derived Representations, Temporal Selection, and Reproducible Prediction

Wes Madrigal
Kurve AI

November 5, 2026

Abstract

Relational prediction depends on how information from linked, timestamped tables is compressed into task-level representations. KurveRSC 0.2.0 selects executable relational feature programs using temporal validation, while GraphReduce materializes their entity-level feature frames. Frozen execution plans preserve the selected representation during prediction. A staged search increases evaluation fidelity, checks finalists under the production fitting procedure, and selects training-support policies and generic optional readouts with a shared admission rule. The downstream learner is pluggable; the evaluated procedure uses CatBoost with point-in-time source visibility. RelArena's maintainers independently reproduced the public packages on 21 classification and regression tasks across seven databases. On the November 5, 2026 combined leaderboard of 16 methods, KurveRSC achieves 1913.4 Elo (95% task-bootstrap interval 1832.3-2050.6), ranking first among registered systems and fully open-source methods, and second overall. These results highlight the importance of optimizing relational representation and signal compression for the prediction task. Representation loss, selection fidelity and computational cost remain limitations.

RelArena-α results at a glance

Table 1: Aggregate-Elo ranks on 21 tasks from seven databases, at seed zero. November 5, 2026 RelArena-α snapshot.

Comparison cohort	KurveRSC rank
Registered systems	**1**
Fully open-source methods	**1**
All models and systems	**2 of 16**

KurveRSC achieves **1913.4 Elo** (95% task-bootstrap interval 1832.3-2050.6). All ranks use point estimates from the same published combined leaderboard. The openness cohort is our classification of complete, locally executable open-source prediction procedures; category definitions are in Section 10.3 [1], [2].

Contents

1 Introduction **3**

Architecture overview **4**

2 Scope and release identity **5**

3 Relational compression as a learned program **6**

4 Temporal and data contracts **10**

5 Representation space **11**

6 Staged graph search and increasing fidelity **13**

7 A symmetric production-faithful finalist check **16**

8 Temporal training policies and the generic optional menu **17**

9 Execution, portability, and production use **18**

10 Maintainer reproduction and release qualification **19**

11 Cost, resources, and parallelization **23**

12 Related work and architectural claims **24**

13 Limitations and next steps **25**

Acknowledgments **25**

References **26**

14 Appendix A: release and implementation provenance **29**

15 Appendix B: complete per-task comparison **30**

1 Introduction

Prediction from relational databases requires converting linked records at different granularities into representations aligned with a target entity and prediction time. Which relationships, historical observations, reductions and annotations survive this conversion can affect predictive quality and cost. KurveRSC treats the compression program as an explicit, task-conditioned object whose quality is measured with the learner that will consume its output.

The original KurveRSC work introduced validation-guided GraphReduce program selection with a downstream learner in the loop and a frozen execution plan for replay [3]. It established a competitive CatBoost-based relational prediction baseline. In the November 5, 2026 combined comparison panel used here, the earlier registration achieves 1773.8 Elo [1]. Version 0.2.0 builds on that foundation with richer bounded representations, staged evaluation, production-faithful finalist checks and temporal training-policy and readout selection.

KurveRSC selects a relational feature program with a supervised learner in the loop. GraphReduce executes that program, reducing timestamped, related records to an entity-level feature matrix; a pluggable downstream learner evaluates and fits the resulting representation. CatBoost is the evaluated reference learner in this report. The staged procedure allocates increasing fidelity to a bounded set of schema-derived candidates, checks finalists with the production fitting lifecycle, and compares temporal support policies and generic optional readouts before saving keyed predictions.

This report specifies the published **KurveRSC 0.2.0 / GraphReduce 1.10.18** procedure.

All reported experiments use a shared staged search and selection procedure, with budget limits and validation rules specified in advance. Graph configurations and optional components are selected using each task's schema and temporal validation data.

The optional menu contains parent text heads, training-frame ensembles, target-domain projection, a zero-aware distributional readout, and object breadth. All optional candidates challenge the same incumbent with one temporal admission rule. Their eligibility depends on declared inputs and capacity, independently of previous admissions. The finalist check treats binary classification and regression symmetrically, using a 1% relative-loss cap on `1 - AUC` and MAE, respectively.

RelArena's maintainers reproduced all 21 RelBench v1 tasks at seed zero with these public packages and merged the system on November 5, 2026. On the frozen 16-method combined leaderboard, KurveRSC ranks **first among registered systems and fully open-source methods, and second overall**, with **1913.4 Elo** (95% task-bootstrap interval 1832.3-2050.6), versus 1933.6 for the latest TabPFN-Rel client. It wins 11 of their 21 task comparisons. This report includes every reproduced per-task score and the successful jobs' measured resources [1].

The central research objective is to characterize how much predictive power can be recovered through relational signal compression and how that value interacts with learner capacity. The entity-level feature frame deliberately separates the relational program from a pluggable downstream learner. As tabular foundation models (TFMs) become more capable, compatible adapters can bring them into the same architecture for selection and replay. The reported scores evaluate the complete CatBoost-based procedure.

The architecture also separates representation from the execution backend. The Kurve product provides production workflows, while the evaluated open-source recipe uses DuckDB. The main open questions concern selection fidelity, representation loss and computational cost.

Code and reproducibility. The source code, installation instructions and reproduction documentation are available at github.com/kurveai/kurversc [2]. The evaluated packages are `kurversc[staged]==0.2.0` and `graphreduce==1.10.18`.

Architecture overview

Figure 1 follows a customer-prediction example from linked tables to a saved prediction procedure. Both rows read left to right. The stages make different decisions about the representation, its fitting policy and optional predictors.

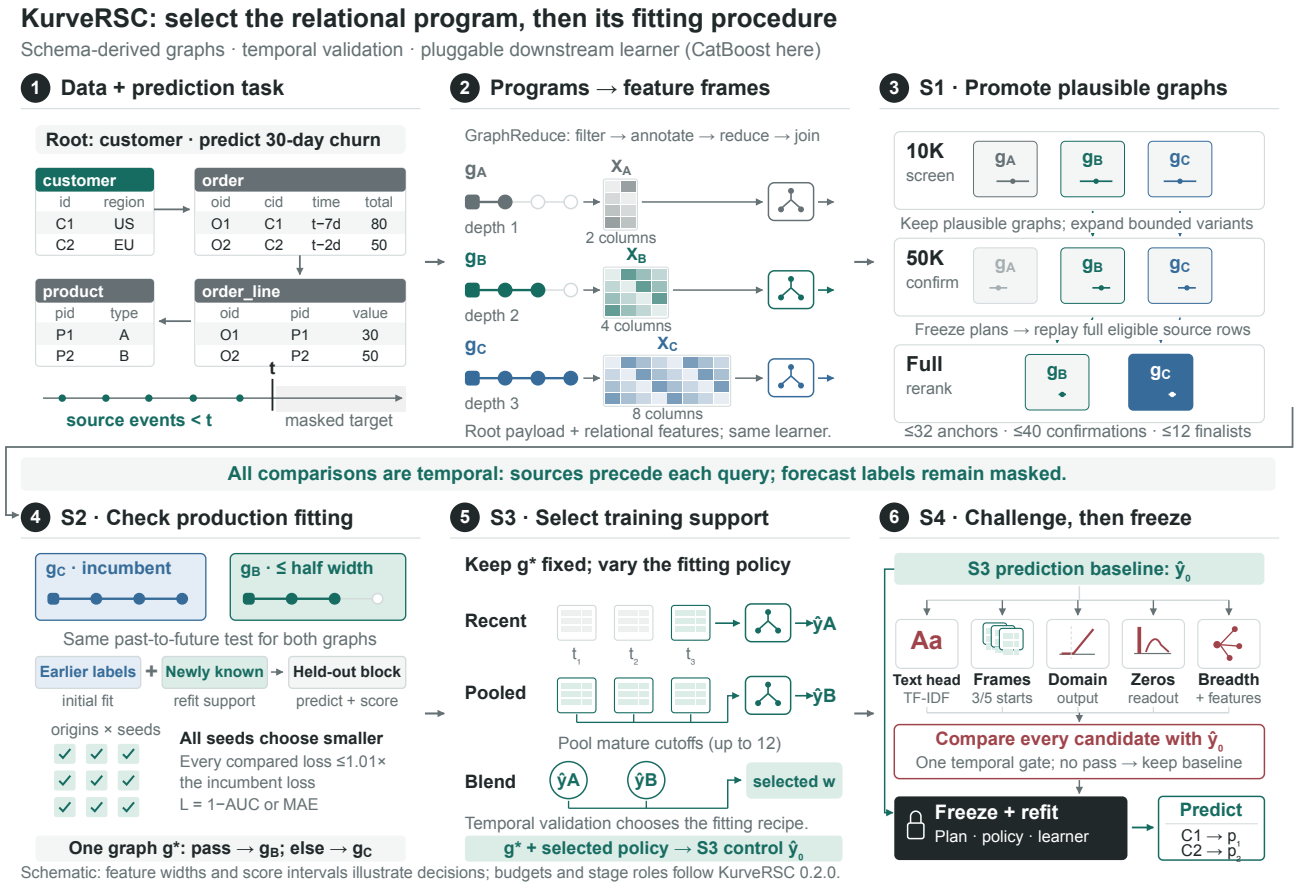


Figure 1: KurveRSC 0.2.0 from linked histories to a frozen prediction procedure. Panels 1-2 construct entity-level feature frames; panels 3-6 show S1-S4. Feature widths and score intervals are schematic, not measured results. Only applicable S4 options run for a given task. The downstream learner is pluggable; CatBoost is the evaluated reference. Each retrospective forecast respects source and label-availability boundaries; outer test scoring remains external.

S1 chooses what information to build. A graph configuration specifies which relationships to follow and how to summarize their records into columns for the target entity. GraphReduce builds those columns, and a learner measures their predictive value. KurveRSC first screens many configurations cheaply, retains plausible alternatives, and spends more data and computation on the survivors. This stage produces a leading graph and evidence about completed finalists.

S2 checks that graph under the production fitting procedure. It compares the leader with at most one substantially smaller completed finalist. Both follow the same historical simulation: the initial fit learns from earlier labeled examples; the refit incorporates later cohorts after their outcomes have become known. This reproduces the fitting lifecycle used at deployment. Both graphs receive matching training cohorts and seeds. The following forecast block stays outside fitting and is used only to score their predictions. S2 retains one graph; it does not repeat the broad search.

S3 chooses which historical examples and fitting policy to use. The graph and its feature definitions now stay fixed. Applying that graph at several past dates creates several groups of labeled examples; the same customer at two dates is two training examples, not two new columns. S3 compares recent initial support with bounded historical pooling and other declared fitting policies. Pooling combines examples before fitting a shared model; a temporal blend combines predictions from complete fitting policies. Every policy includes the observed-label refit lifecycle. The selected graph-plus-policy becomes the baseline for S4.

S4 tests optional prediction procedures against that baseline. Its "shared control" is simply the selected S3 procedure and its predictions on the same validation cases. Some options introduce additional signal: the parent-text head builds a separate TF-IDF word representation and predictor, and object breadth adds summaries of distinct objects and interaction concentration to its challenger model. Other options combine frame models, restrict predictions to the observed target domain, or model zero outcomes separately from positive amounts. Eligible raw root fields, including strings handled categorically, can already enter S1; S4 is where this release separately tests the lexical TF-IDF head. The frame-ensemble options use three or five initial historical frames, plus the declared observed-label refit cohorts, fitting separate frame members and combining their predictions. Each option independently competes with the same S3 baseline under one admission rule. At most one optional recipe is retained; if none passes, the baseline remains. The final procedure is frozen and fitted for deployment. Validation seeds test repeatability; they do not imply a seed ensemble at inference.

Why S4 is separate. We postpone these optional procedures to reduce search cost: evaluating every extra representation, model and readout on every candidate graph would multiply expensive fits during screening and confirmation. This is a budgeted architectural choice. It does not establish that the best graph under the initial learner is also best with every optional procedure. A graph rejected earlier might work better with a text head or extra object features. Future versions may place compatible feature generators earlier in representation search, or compare downstream recipes across several full-fidelity graph finalists. Those alternatives require controlled evaluation of both predictive quality and computation.

2 Scope and release identity

This document describes the fixed staged procedure rather than every native KurveRSC option. Its software dependencies are the public packages `kurversc[staged]==0.2.0` and `graphreduce==1.10.18`. The corresponding source identities are recorded in Appendix A for audit; installation uses package versions, not a Git dependency. The mathematical formulation, representation space, selection procedure, data boundaries and reproduced results are specified in this report.

2.1 Procedure at a glance

Area	Published generic procedure
Core stages	Schema-derived graph search, full-fidelity rerank, production-faithful finalist check, temporal training-policy selection.
Generic optional menu	Text, frame ensembles, target-domain projection, zero-aware distributional readout, object breadth.
Finalist symmetry	The same bounded comparison for MAE and binary ROC-AUC.
Optional admission	One selector, shared origins/seeds and incumbent, independent applicability, menu-size multiplicity adjustment.
Source boundaries	Explicit strict event cutoffs, separately declared availability, phase-censored task contexts and verified replay bindings.
Evidence	Maintainer reproduction of all 21 tasks, published combined leaderboard, per-task scores and measured job resources.

The fixed recipe retains globally specified model sizes, blend menus and capacity limits. They are shared defaults disclosed below, not values rediscovered on each database. The procedure applies to new schemas through their declared tables, keys, root entity, horizon and observed labels; candidate selection uses the task's own temporal validation evidence.

2.2 Interfaces

Interface	Responsibility
`kurversc.fit()` / `predict()`	Native graph search, fitting, and frozen-plan replay. The caller chooses settings.
`kurversc.run_staged_task()`	The complete fixed staged procedure described here.
RelArena registered system	Converts harness-owned splits into the task context, invokes the packaged procedure, and returns keyed predictions for external scoring.

Calling native `fit()` with its defaults does not by itself reproduce every stage of the submitted benchmark procedure. The RelArena system name is `kurversc-2026-10-22`; the date is a registry identifier.

3 Relational compression as a learned program

3.1 Task, configuration, and learning objective

A task declares a database D, a root table r, a target definition, a forecast horizon H, and a metric. A prediction request (e, t) asks for the outcome of root entity e after query cutoff t, under that target definition. For example, a customer task can ask whether an order will occur in the next 30 days. A *feature frame* is a matrix with one row per requested entity at one cutoff. Historical frames repeat this construction at earlier cutoffs; they are separate supervised examples, not additional columns.

The database declares primary and foreign keys, source clocks, and observed labels. KurveRSC compiles these relationships into a graph oriented outward from the root. A graph node is a *path occurrence* of a table: the same physical table can occur more than once under different aliases. An edge records both join keys and whether its related rows require reduction. The direction of this compute edge describes feature flow toward the root; it need not match the direction in which the database stores its foreign key.

For the bounded search, write a configuration as

$$g = (d, \mathcal{F}, a, h, o, c; B).$$

Here d is maximum path depth; $\mathcal{F}$ contains `base` and the selected feature families; a enables or disables automatic annotation; h controls the historical-target relation; o bounds occurrences of a base table along a path; and c specifies categorical handling, including bounded tie refinement. B contains payload and feature-expansion limits. The schema and these choices determine the executable paths. The staged search does not independently search every possible edge subset, window length, numeric transform, and learner hyperparameter. CDF sketches are fixed on in the evaluated profile.

GraphReduce executes the resulting feature program:

$$z_{e,t} = \Phi_g(D_{<t}, e, t), \qquad \hat{y}_{e,t} = f_\theta(z_{e,t}).$$

The notation $D_{<t}$ abbreviates the visibility contract in the next section; a source record may also have an independent availability clock. Let A be the declared learner-fitting procedure, including its row budget, seed, and temporal support policy. For each temporal validation origin k, it produces

$$\theta_{g,k} = A(Z^{\text{train}}_{g,k}, Y^{\text{train}}_k), \qquad s_{g,k} = M\left(Y^{\text{val}}_k, f_{\theta_{g,k}}(Z^{\text{val}}_{g,k})\right).$$

Each row of either Z is materialized at that row's own query cutoff. Earlier training targets must be observable by the fitting boundary. M is ROC-AUC for binary classification or MAE for regression; the learner itself optimizes Logloss or MAE, respectively. Search compares g under the same declared fitting procedure, using these validation scores and the retention rules in Section 6. It does not differentiate through SQL, learn table embeddings, or select on held-out test scores. This is supervised selection of a relational program with a learner in the loop.

3.2 Keyed reduction and propagation

Consider a parent occurrence p and a related occurrence v. Let X_p denote the parent's visible rows and payload, and let Z_v include the child's own payload plus features already received from its descendants. If many child rows refer to one parent, a reduction $R^g_{v\to p}$ groups Z_v by the declared child foreign key and computes a bounded vector of summaries. GraphReduce then left joins this one-row-per-key result to the parent:

$$Z_p = \mathrm{LeftJoin}_{k_p=k_v}\left(X_p, R^g_{v\to p}(Z_v)\right).$$

The join is repeated for each child. A keyed dimension lookup is different: when the parent's foreign key identifies a single related row, that row's visible attributes can be joined without aggregating it. The adapter records this distinction in the edge's `reduce` flag. Keys must satisfy the declared uniqueness and cardinality contracts; unrestricted many-to-many joins are not a substitute for reduction.

Edges are processed in reverse depth-first order, so descendant features exist before their receiving table is itself reduced. For `customer -> order -> line`, line features first become order features; those features then become customer features. Summarizing each child before joining also prevents sibling fan-out: independently joining orders and support tickets would otherwise multiply rows and distort counts and sums. Under valid key contracts, a reduction and its join preserve the receiving table's grain.

Figure 2 distinguishes a reducing edge from a dimension lookup. Each summary joins at the receiving grain before it is available to the next reduction.

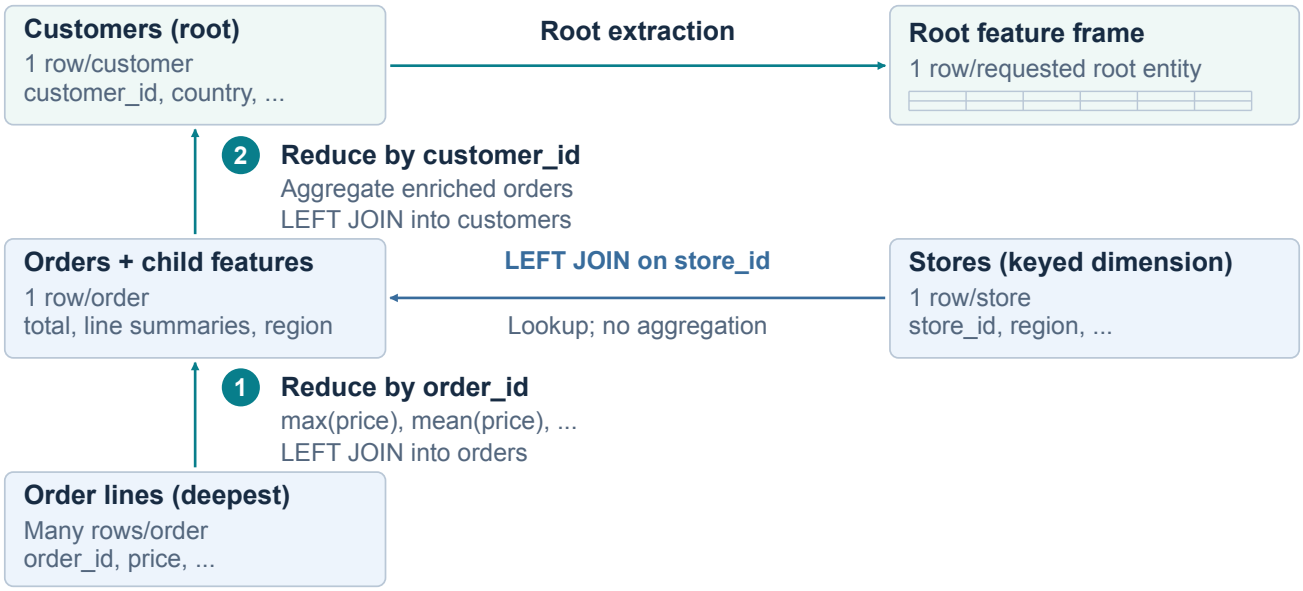


Figure 2: Keyed reduction and propagation toward the root. Arrows show feature flow: line summaries first enrich orders, then enriched orders are summarized for customers. A store dimension uses a unique-key lookup without aggregation. All inputs obey the phase's temporal contract; extraction preserves the requested root grain.

Algorithm 1 Materializing a relational program at one cutoff

Input: Program g; phase-censored D; cutoff t; root requests Q; optional replay plan P_0.
Output: Root payload Z and a frozen or verified execution plan P.

$\mathcal{G} \leftarrow \textsc{InstantiatePaths}(D, g, P_0)$ ▷ Restore and verify replay bindings
$X \leftarrow \textsc{LoadSelectAndAnnotate}(D, g, t, P_0)$
for each edge (p, v) in reverse depth-first order of $\mathcal{G}$ **do**
  if the edge requires reduction **then**
    $S_v \leftarrow \textsc{WindowedReduce}(X_v, k_v, g, t, P_0)$
  else
    $S_v \leftarrow \textsc{KeyedLookup}(X_v, k_v)$
  end if
  $X_p \leftarrow \textsc{LeftJoin}(X_p, S_v;\ k_p = k_v)$
end for
$Z \leftarrow \textsc{AlignRootPayload}(X_r, Q)$ ▷ Separate targets and routing
$P \leftarrow \textsc{FreezeOrVerify}(P_0, \mathcal{G}, \text{operations}, \text{schema})$
return (Z, P)

Unavailable payload remains missing; a missing child group is not universally imputed to zero. Request keys and order are retained throughout.

The procedure executes SQL through the backend; it does not need to construct one fully joined raw table in Python. The final frame can nevertheless be wide or expensive to materialize. A materialized aggregate is a new payload column at the next level, not a retained bag of its original rows. For columns with recognized aggregate lineage, the recipe permits one compatible continuation function: maxima continue with `max`, sums and counts with `sum`, and averages with `avg`. This controls expansion but also determines which distinctions survive [4].

3.3 Worked three-table example

Suppose the root table contains customers A, B, and C with a known `country` attribute. The following small database illustrates two graph configurations; it is not a benchmark or an additional stage. All timestamps are in January 2026, and the request cutoff is **January 11 at 00:00**. Orders and lines have strict source clocks. The target is a future 30-day order indicator, supplied separately as observed labels when it becomes available.

Order	Customer	Time	Total
O1	A	Jan 7, 00:00	20
O2	A	Jan 10, 00:00	100
O3	B	Jan 9, 00:00	60
O4	A	Jan 11, 00:00	999

Line	Order	Time	Price
L1	O1	Jan 7, 00:00	10
L2	O1	Jan 7, 00:00	10
L3	O2	Jan 10, 00:00	100
L4	O3	Jan 9, 00:00	60
L5	O4	Jan 11, 00:00	999

The keys are `orders.customer -> customers.customer` and `lines.order -> orders.order`. O4 and L5 are invisible at this cutoff, even though their dates match the query date. A depth-one `base` configuration sees orders and can produce the first three summaries below. A depth-two `base` configuration also reduces lines to orders and propagates the resulting summaries. Hold annotation and target history off in both configurations to isolate depth; CDF remains enabled but is omitted from this displayed subset.

Customer	Order count	Total sum	Total mean	Line-price max	Line-price mean of means
A	2	120	60	100	55
B	1	60	60	60	60
C	NA	NA	NA	NA	NA

The last two columns require depth two. At order grain, O1 has line-price maximum 10 and mean 10, while O2 has maximum 100 and mean 100. Customer A then receives $\max(10, 100) = 100$ and $(10 + 100)/2 = 55$. These descriptive column names stand for path-qualified operation aliases; the table displays selected features, not every generated feature. Root `country` is also available to the learner, while customer/order identifiers route computation and are removed from its raw payload.

This example exposes two consequential choices. First, joining raw lines to orders before summing totals would count O1's total twice, incorrectly giving A a total of 140 instead of 120. Reducing first avoids that error. Second, the mean of per-order means is 55, whereas the mean over A's three lines is 40. The default continuation does not silently recover an item-weighted mean. Different programs preserve different information, and deeper is not always better. Temporal validation decides whether a representation is useful for the declared target.

Figure 3 traces the reductions for customer A. The numerical example displays selected features, with category and CDF details omitted for clarity.

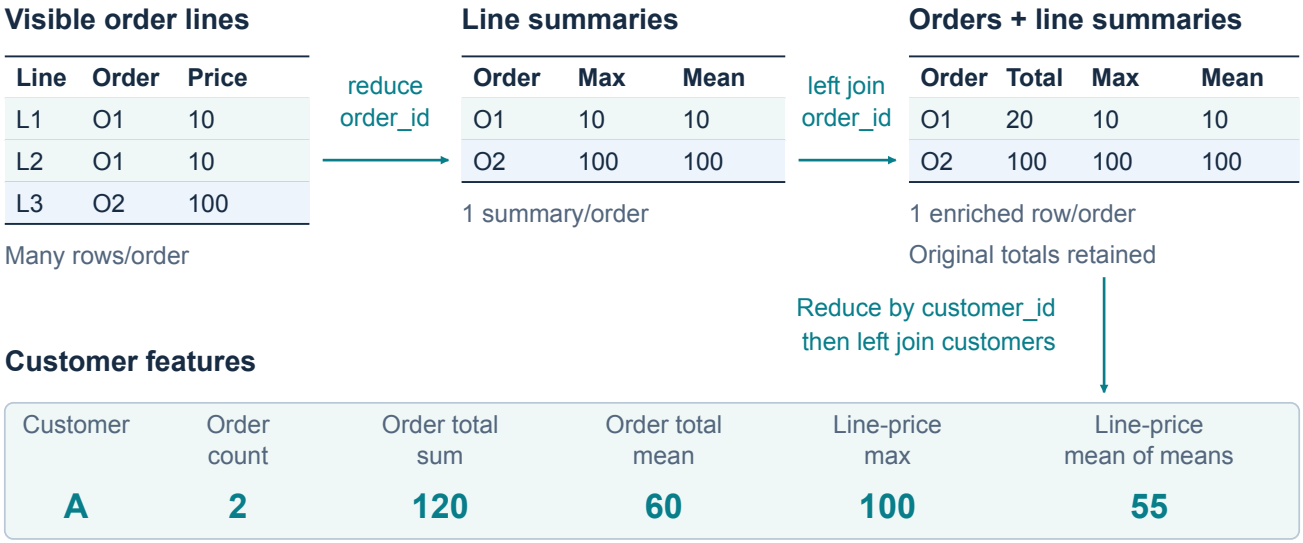


Figure 3: Worked example at customer grain. Per-order line maxima and means join to the original orders before customer aggregation. Customer A's order totals are counted once, giving 120. The propagated mean of order means is 55; the mean over all three lines is 40, illustrating the weighting implied by this program. Same-cutoff source rows are excluded.

3.4 The learner boundary

Training defines the learner's column order and types. Entirely missing and constant training columns are removed; categorical values use string encodings with a missing-value token, datetimes become numerical timestamps, and nonfinite numeric values become missing. Prediction applies that saved schema and preprocessing rather than selecting columns again. Thus the artifact contains both the relational execution plan and the learner-facing schema.

The matrix is an interface to the learner. Information can be lost in the chosen representation, but a matrix interface does not imply that the features were computed from one original table. Conversely, adding columns cannot recover distinctions already erased by a reduction. Two entities with the same means and counts may have different event ordering, category-conditioned outcomes, or neighbor structure. The families below preserve some of these distinctions within a finite program budget.

The research focus is the predictive information preserved by the relational program and its interaction with the learner. Within matched graph comparisons, KurveRSC varies the representation under a declared downstream fitting policy. The reported end-to-end scores measure the complete pair; separating the contributions of compression and learner capacity requires controlled comparisons of both components.

The downstream learner is intentionally replaceable at the typed frame boundary. Native `model_backend` and `final_model_backend` interfaces already include tabular foundation-model adapters. More capable TFMs can be integrated without replacing the relational-program abstraction. Their use should evaluate candidate programs with the intended learner and the same temporal contracts; a graph selected for one learner need not be optimal for another.

The benchmark learner is CatBoost [5]. The fixed staged procedure evaluated here uses CatBoost for graph search and final prediction; native learner integrations are separate interfaces.

4 Temporal and data contracts

4.1 Source time and knowledge time

Each source row is checked against three timestamps:

- **Event time** (`event_time`): when the event represented by that row occurred.
- **Query cutoff** (`query_cutoff`, denoted t elsewhere): the timestamp for which a prediction is requested.
- **Phase cutoff** (`phase_cutoff`): the boundary of the frozen source snapshot supplied to the current training, validation or prediction phase.

For a source declared strictly timed, the event must precede both cutoffs:

$$\text{event_time} < \min(\text{query_cutoff}, \text{phase_cutoff}).$$

In words, the row's event must occur before both the prediction cutoff and the phase's frozen data boundary. An event exactly at either boundary is excluded. For example, with a query cutoff of January 11 at 00:00 and a phase cutoff of January 10 at 00:00, an event on January 10 at 12:00 is excluded: it precedes the query but falls after the phase boundary.

Some sources also declare a separate **availability time** (`availability_time`): when the forecasting procedure could first observe the row. The phase export additionally requires

$$\text{availability_time} \leq \text{phase_cutoff}.$$

This availability comparison is inclusive; the event-time comparisons remain strict. Advancing the query cutoff does not advance the frozen phase boundary. RelBench timed-table adapters declare strict event boundaries. An explicit strict boundary cannot be relaxed by a legacy compatibility offset.

Filtering precedes annotation, category discovery, and feature materialization. The task context creates censored phase exports before provider discovery. Prediction contexts contain request keys and permitted source data, without held-out answers. Validation truth is used at the declared validation phase; held-out test scoring happens outside the prediction procedure after selection and keyed predictions are saved.

A source without clocks must be explicitly declared timeless. That declaration is a caller responsibility, not an inference that an undated mutable table was historically available. A source clock also does not reconstruct past versions of overwritten values. Correct point-in-time inputs remain necessary.

4.2 Historical target relations and peer context

Observed targets can provide useful prior state when the entity key is removed from the learner. KurveRSC can expose mature training labels as a bounded child relation. A label's query timestamp being earlier than the current request is necessary but insufficient: its prediction horizon must have matured, and any declared label availability must satisfy the relevant training boundary. Held-out targets are never added to this relation.

The history provider reads eligible observed labels independently of any sampling of the learner's training examples. A limit on the examples used to fit the learner therefore does not itself truncate the label-history source. The staged recipe separately bounds peer-history discovery to 50,000 history rows. Counts, means, recency, and peer

summaries are therefore bounded representations rather than a promise to expose every historical observation in every provider.

Generic peer context uses declared relationships and visible history. It can summarize other entities attached to common objects while excluding the current root's own contribution where the peer definition requires it. Peer-label availability may be stricter than the general source-export rule; the provider applies its own conservative visibility checks.

The graph's history switch controls its explicit historical-target child table. The staged profile's configured peer provider is separate: disabling that child does not disable every peer-label summary. Both must obey their declared availability contracts.

4.3 Request preservation and replay

A strict cutoff can hide the root row itself, for example when it was created exactly at the request time. The forecast request remains valid: its key and position are retained, and unavailable payload becomes missing. Dropping such requests would change the evaluation cohort.

A frozen plan records source bindings, path instances, operations, feature schema, category choices, and applicable numeric cutpoints. Prediction restores and verifies those bindings rather than rediscovering a graph. The source superset must support every offered repeated-table configuration. The current implementation corrects a mismatch between the offered path space and the source graph and checks restored bindings during replay.

5 Representation space

5.1 GraphReduce feature families

The fixed procedure builds candidates from applicable families and declared schema relationships. The following descriptions summarize GraphReduce's feature families [4].

Family	Signal represented
`base`	Standard numeric and categorical reductions, with bounded numeric distribution sketches in this recipe.
`temporal`	Numeric magnitude, dispersion and change within historical windows.
`state`	Latest values, ordered lags and summaries of the last several events, useful for irregular event cadence.
`sequence`	Activity rates, recent shares, concentration, bursts and active spans.
`conditional`	Counts, shares, presence and change under bounded categorical or annotated conditions.
`episode`	Event opportunity through row and distinct-event counts, including windowed counts.
`context`	Peer-group size and numeric deviations computed on child rows before reduction.

GraphReduce also accepts caller-declared semantic expressions and predicates. These are available natively; the fixed staged recipe does not automatically invent arbitrary semantic expressions. Parent/root payload and applicable annotations remain available, subject to the same source visibility and payload budgets.

These families specify operations rather than semantic roles assigned to a particular database. For an entity's visible event rows, `temporal` can compare the mean amount in a recent window with an older window; `state` can retain the latest amount and the mean of the last three events; and `sequence` can describe event counts per day and recent-to-longer-window activity shares. `conditional` can count events with a selected status and measure that status's share over time. `episode` distinguishes row opportunities from distinct event keys. Given a declared group key, `context` can first annotate each child row with its group size and $x - \text{mean}(x \mid \text{group})$, then reduce those deviations to the root. It therefore retains information unavailable from the root's unconditional mean alone. These are representative expressions; applicability and payload limits determine which are emitted.

The families overlap. `base` already includes dated activity/recency and some bounded categorical trajectories, as well as standard type-dependent aggregations. Adding `sequence` does not create time awareness from nothing; it adds a richer bounded collection of activity summaries. The planner audit and saved operations, rather than the family name alone, identify the actual feature representation.

5.2 Automatic annotation before aggregation

Annotation creates row-level columns that later reductions can summarize. For a categorical `status`, a retained value such as `closed` can yield $\mathbf{1}$[status = closed] and a numeric payload such as `amount` can yield `CASE WHEN status = 'closed' THEN amount END`. Summing or averaging the indicator measures frequency or share; averaging the gated amount measures magnitude conditional on that status. A global mean and a status count cannot in general recover this cross-column association.

The released procedure discovers these choices on eligible training source samples. Its node settings allow at most ten categorical annotation columns, four numeric columns for gated values, and three category values per gated numeric column. Ordinary category retention uses a cardinality threshold of 20 and a top-five policy; the compatible anchor can retain all low-cardinality values and top categories above the threshold. Category frequencies and tie choices are frozen with the plan. Forecast data does not fit a new category vocabulary. Source-column, per-column expansion and propagation caps still apply, so annotation does not imply an unbounded Cartesian product.

This operation also runs on the root/parent node. Retained parent payload, eligible categorical indicators and gated numeric annotations can enter the learner directly; child annotations can first travel through aggregation. A dated root can contribute an age-at-cutoff feature. Automatic text-shape annotation is disabled in the evaluated graph recipe; the separately validated TF-IDF parent-text head in Section 8 is a different operation. Automatic annotation does not infer an outcome's meaning or invent arbitrary SQL predicates.

5.3 CDF sketches inside `base`

For a numeric payload v on a reducing edge, let $R(e,t)$ be the related source rows assigned to receiving entity e at query cutoff t, after path/key matching, historical-window filtering (Section 5.4) and phase-visibility filtering (Section 4.1). The finite-value support and its size are

$$R_v(e,t) = \{r \in R(e,t) : v_r \in \mathbb{R}\}, \qquad n_v(e,t) = |R_v(e,t)|.$$

Here v_r is the value of v in row r; missing values, NaN and either infinity are excluded from $R_v(e,t)$. Training-only quantile cutpoints $c_{v,j}$, with $j \in \{1, \dots, J_v\}$ and $0 \le J_v \le 7$, define the bounded empirical CDF features:

$$\widehat{F}^{(v)}_{e,t}(c_{v,j}) = \begin{cases} \dfrac{1}{n_v(e,t)} \displaystyle\sum_{r \in R_v(e,t)} \mathbf{1}\{v_r \le c_{v,j}\}, & n_v(e,t) > 0, \\ \text{missing}, & n_v(e,t) = 0. \end{cases}$$

Thus each feature is the fraction of finite values at or below its cutpoint; empty finite support produces a missing value, not zero. GraphReduce fits the cutpoints from at most 20,000 eligible training rows using observed-value quantiles, deduplicating the cutpoints and discarding any equal to the calibration maximum. It also records finite-value coverage separately. Columns with fewer than three distinct finite calibration values emit no sketch. Cutpoints are frozen in the execution plan and are not refitted on validation or forecast data.

CDF is part of `base`, not an additional family. **The fixed staged recipe sets `base_cdf=True`.** Native and diagnostic interfaces can compare CDF on/off, but the published recipe does not double its entire graph menu into both forms. A bounded sketch preserves some distributional differences; it does not retain the complete empirical distribution or every cross-column dependency.

5.4 Historical windows versus forecast horizon

The forecast horizon H defines the target and label maturity. The compute lookback L limits source history; these are different quantities. The native materializer's default lookback used by this profile is 3,650 days. A timed reduction uses source rows in $(t - L, t)$, intersected with the phase's permitted data. A window of w days further requires event_time $\ge t - w$; it never restores a row excluded by the strict upper cutoff or the phase boundary.

The evaluated profile disables inferred periods. GraphReduce's default day grid is $\{1, 3, 4, 7, 14, 30, 60, 90, 180, 365, 730\}$, with the longer compute lookback added during graph hydration. The expansion budget can retain only a subset of the corresponding expressions. These durations specify retrospective windows, not rounding of timestamps: a query at 15:30 still excludes source rows at or after 15:30. `state` instead orders eligible rows by event time and uses row-relative lags and trailing windows, including 3, 5 and 10 events. Learned plans preserve their period choices across replay. Optional providers can impose separate lookback limits, such as object breadth's 730-day cap.

5.5 Depth, repeated tables, and payload limits

Candidates can reach depth four. An occurrence limit of one gives paths without repeated base tables; a limit of two admits bounded self/peer constructions. The executable graph remains a DAG because repeated tables have distinct path instances. This permits bounded walks through a cyclic relational schema without unbounded traversal.

The staged source-column budget is four payload columns per family/node, up to 32 derived features per selected source column, and one continuation function per propagated derived column. Structural keys, dates and context keys remain outside the payload budget. Deterministic column ranking uses coverage, entropy, spread, cardinality and identifier redundancy. The feature audit stores utility rank, selection rank, strategy and budget eligibility.

After promotion, categorical refinement tests bounded tie choices for up to two parent shapes. Available tie seeds are 0 through 7; the stage permits eight evaluations in total. These are distinct from the three model seeds used in procedure validation. Modern high-cardinality handling and a compatible anchor provide bounded alternatives; no payload policy exposes an unlimited set of identifiers or free-text features.

6 Staged graph search and increasing fidelity

Exhaustively fitting all depth, family, annotation and topology combinations at full fidelity is expensive. The `staged-retention` profile evaluates a bounded schema-derived menu, promotes survivors, then increases data fidelity. Search uses no saved task-to-winning-configuration lookup.

6.1 Candidate construction

The initial anchor lattice contains `base` plus subsets of `temporal`, `sequence`, `conditional`, and `episode`, under both annotation policies and bounded topology variants. It reaches depth three, with the more expensive family combinations initially limited to depth two. Capability checks remove unavailable operations, and a deterministic balanced collection selects at most 32 candidates. It reserves base controls and covers family, annotation, depth and topology strata before filling its budget. This is a bounded sample of a potential program space, not exhaustive search over every schema path.

Up to six successful anchors seed expansion, retaining the complexity-aware recommendation among them. Challengers vary depth through four, repeated-table limits, target history, and additions of `state` or `context`. A separate sparse seed panel includes `base + state`, `base + context`, and their interaction even if plain `base` was not promoted. This avoids requiring a useful small modern graph to descend from a large winning anchor. Combined challengers are added when individual additions show meaningful validation gains over the same parent. Depth restrictions on some expensive expansions and hard stage budgets still apply. Unavailable or estimated-overwidth programs are pruned before execution; duplicate configurations are evaluated only once per relevant stage.

Stage	Max.	Purpose
Screening	32	Initial comparison, nominally 10,000-example frames.
Family/topology challengers	24	Applicable alternatives around retained candidates.
Combined challengers	12	Bounded combinations of promising choices.
Confirmation	40	Higher-fidelity comparisons, nominally 50,000-example frames.
Historical promotion	8	Additional eligible historical-cutoff evidence.
Categorical refinement	8	Deterministic categorical tie alternatives.
Full rerank	12	Frozen-plan replay on full eligible source frames and up to three recent cutoffs.

These ceilings sum to 136 stage evaluations. They are neither a count of unique graphs nor a count of CatBoost fits. Applicability, duplicate configurations, available support and stage outcomes can reduce evaluations. Multiple cutoffs, seeds, temporal policies and optional heads can increase learner fits beyond that sum.

6.2 Retention, promotion, and final graph selection

The search makes two different decisions: which graphs receive more evaluation, and which fully evaluated graph is selected. Both use validation evidence.

Keeping plausible graphs for more evaluation. Let g be a candidate graph, s_g its validation score at the current evaluation stage, and SE_g the estimated sampling standard error of that score. To compare both metrics in the same direction, define a higher-is-better utility:

$$U(s) = \begin{cases} s, & \text{for ROC-AUC,} \\ -s, & \text{for MAE.} \end{cases}$$

For example, AUC 0.80 has higher utility than AUC 0.79, while MAE 4 has higher utility than MAE 5 because $-4 > -5$. A trial also records its feature count, frozen plan and cutoff evidence.

Let g_* denote the current leader and δ the practical retention margin: 0.01 AUC for classification, or 1% of the leader's MAE for regression. For single-fold evidence, a candidate belongs to the confidence frontier when

$$U(s_{g_*}) - U(s_g) \le \max\left(\delta, \sqrt{\mathrm{SE}_{g_*}^2 + \mathrm{SE}_g^2}\right).$$

The left side is the candidate's observed loss relative to the leader. The right side is the larger of the practical margin and the two scores' combined estimated standard error. The rule therefore keeps a graph close enough in score, or uncertain enough, to justify more evaluation. For example, with an AUC leader at 0.800, a graph at 0.794 belongs to the frontier because its gap of 0.006 is below 0.01. When a trial already combines multiple cutoffs, its recorded stability-adjusted objective, defined below, supplies the utility term in this same rule.

Frontier membership does not guarantee promotion: if the frontier exceeds the stage budget, structural diversity chooses a bounded subset. Otherwise up to four additional structural controls can fill spare slots. Diversity retains representatives of family interactions, sparse family controls, annotation policies, depth, history and topology alternatives. An unresolved low-fidelity tie thus preserves several hypotheses, subject to capacity; it does not establish equal full-fidelity performance. The 0.01-AUC *retention* margin is distinct from S2's 1% *relative ranking-loss* limit.

Nominal 50K confirmation replays each survivor's frozen plan. A historical promotion stage runs only if additional eligible training frames are available. Categorical refinement gives up to two score-leading structural parents a bounded number of deterministic category-tie variants. Before full rerank, the best observed variant of each categorical lineage represents that graph, so many tie seeds cannot consume every finalist slot. Confidence-frontier retention then selects at most twelve lineages for full-source comparisons.

Ranking graphs after full-source evaluation. At full rerank, all candidates use the same available chronological validation folds. A fold is an earlier prediction origin, not a random split. Let $K \in \{1, 2, 3\}$ be the number of available folds, ordered from oldest to newest, and let $s_{g,k}$ be graph g's ROC-AUC or MAE on fold k. Define its mean score $\bar{s}_g$ and across-fold dispersion d_g explicitly:

$$\bar{s}_g = \frac{1}{K}\sum_{k=1}^{K} s_{g,k}, \qquad d_g = \sqrt{\frac{1}{K}\sum_{k=1}^{K}(s_{g,k} - \bar{s}_g)^2}.$$

The ranking score is

$$Q(g) = U(\bar{s}_g) - 0.25\,\frac{d_g}{\sqrt{K}}.$$

Higher $Q(g)$ is better. The first term rewards mean predictive performance; the second subtracts a penalty for variation across time. Here d_g is the population standard deviation, matching the released implementation, and $d_g/\sqrt{K}$ is its heuristic estimate of uncertainty in the mean. The fixed coefficient 0.25 controls this penalty; it is not a confidence level. For equal mean scores, this rule favors the graph with less variation across folds. With only one fold, $d_g = 0$ and this term provides no stability evidence. $Q(g)$ is used to select graphs; reported task performance uses the original metrics.

Selecting the graph with recent-performance and complexity safeguards. The selector applies these rules in order:

1. Find the leader on the newest fold ($k = K$). Exclude a candidate if the leader's recent gain exceeds both the practical veto (0.01 AUC or 1% relative MAE) and their combined estimated standard error on that fold.

2. Among the remaining candidates, choose the graph with the highest $Q(g)$.
3. A graph with at most half as many features may replace that winner only within a practical, uncertainty-based tie. Paired fold differences estimate uncertainty where available; the permitted mean-score loss is always capped at 0.01 AUC or 1% MAE. Stable configuration ordering resolves remaining ties.

This is the native graph-selection heuristic. S2 subsequently tests at most one smaller finalist using the stricter, symmetric production comparison in Section 7.

Search standard errors are heuristic: MAE uses the standard error of absolute errors, while ROC-AUC uses a positive/negative-count approximation. They do not model all dependence between repeated observations. The optional-head gate later uses paired entity-cluster resampling; these are different statistical operations, not interchangeable confidence guarantees.

6.3 End-to-end search and deployment

The following pseudocode summarizes the published control flow. `Evaluate` includes point-in-time materialization, fitting under the declared learner budget, validation and checkpointing. The phase data D, task $\mathcal{T}$, observed labels Y and per-fit learner budget B are shared inputs to every evaluation and fitting procedure below. Named parameters distinguish row budgets from limits on configurations, evaluations, parents and folds.

The helpers abstract the implementation's stage-specific trial lists. The algorithm separates the evidence ledger E from the active successful trials V. Every evaluation helper returns an augmented ledger, retaining earlier records and their fidelity/stage identities. Confirmation and historical promotion replace V with the newly successful higher-fidelity trials; categorical refinement adds successful variants while retaining their parents. The cap, deadline and capacity checks are internal to each procedure. If a stage is unavailable or completes no usable trial, it retains the preceding active trials. Earlier ledger entries are not mixed back into a later-fidelity comparison merely because they had better cheap scores.

Algorithm 2 Bounded staged relational program selection

Input: Phase-censored D; task $\mathcal{T}$; observed labels Y; masked requests Q; learner budget B.
Output: Frozen artifact A and keyed predictions $\widehat{Y}_Q$.

Representation selection
$C \leftarrow \textsc{BalancedAnchors}(D, \mathcal{T};\ \text{max configs} = 32)$
$(E, V) \leftarrow \textsc{Evaluate}(\emptyset, C;\ \text{rows} = 10\text{K})$
$P \leftarrow \textsc{RetainAnchors}(V;\ \text{max configs} = 6)$
$(E, V) \leftarrow \textsc{EvaluateChallengers}(E, V, P;\ \text{max evals} = 24)$
$(E, V) \leftarrow \textsc{EvaluateEligibleCombinations}(E, V;\ \text{max evals} = 12)$
$F_1 \leftarrow \textsc{FrontierWithControls}(V;\ \text{max configs} = 40)$
$(E, V) \leftarrow \textsc{Confirm}(E, V, F_1;\ \text{rows} = 50\text{K},\ \text{max evals} = 40)$
if $\textsc{HasAdditionalMatureHistory}(D, Y)$ **then**
    $(E, V) \leftarrow \textsc{PromoteHistory}(E, V;\ \text{max evals} = 8)$
end if
$(E, V) \leftarrow \textsc{RefineCategoryTies}(E, V;\ \text{max parents} = 2,\ \text{max evals} = 8)$
$F_2 \leftarrow \textsc{BestPerLineage}(V)$
$G \leftarrow \textsc{FrontierWithControls}(F_2;\ \text{max configs} = 12)$
$(E, g) \leftarrow \textsc{FullSourceRerank}(E, G;\ \text{max folds} = 3)$
Production selection
$g \leftarrow \textsc{CheckSmallerFinalist}(E, g;\ \text{max alternates} = 1)$
$\pi \leftarrow \textsc{SelectTemporalPolicy}(g)$
$q \leftarrow \textsc{SelectOptionalReadout}(g, \pi)$
Freeze and predict
$A \leftarrow \textsc{FreezeAndRefit}(g, \pi, q)$
$\widehat{Y}_Q \leftarrow \textsc{KeyedPredict}(A, Q)$
return $(A, \widehat{Y}_Q)$

Here C is the anchor configuration set; E the accumulated evidence ledger; V the active successful trials; and P the retained expansion parents. F_1 is the frontier with controls sent to confirmation, F_2 contains the best eligible trial per categorical lineage, and G is the full-rerank finalist set. `EvaluateChallengers` includes the sparse controls

described in Section 6.1. `FullSourceRerank` returns updated evidence and one selected graph g, using the ranking and safeguards of Section 6.2, rather than returning a ranked list.

The policy π specifies training support. Every optional readout is compared with the same incumbent defined by (g, π). A returned $q = \emptyset$ means that none was admitted; `FreezeAndRefit` then refits that incumbent without an additional readout. The `rows` values are nominal sampling budgets; `max configs` limits a proposed or retained set, while `max evals` limits new evaluations. Predictions are saved before any external test scoring.

Initial search support uses the smallest recent window of 3-12 available cutoffs with at least 256 labels and, for classification, at least 24 examples of each class when such support exists. Fewer available cutoffs or insufficient support limit what can be compared. These support counts concern examples used to evaluate a representation; they do not automatically set the deployed training policy.

Low-fidelity sampling can affect the connected source cohort as well as the examples scored by the learner. A 10K screening frame is therefore not always a cheap, otherwise identical estimate of the full-source graph. Rare categories and long relational paths can change. Frozen operations are replayed during promotion; full rerank removes the source-cohort sampling cap for its eligible frames. The downstream training-row limit remains separate.

The staged recipe caps graph features at 10,000 and uses a 100,000-row learner fit limit. The fixed CatBoost budget is 300 iterations, depth six, learning rate 0.05 and L2 regularization 10. These bounds control individual work items; they do not bound total materialized source data or total process-tree RAM.

Search support is distinct from deployment support. The initial production policy remains `full_training_frames=1`, even when search or rerank evaluates several historical cutoffs. Wider production support must win as a complete training-policy candidate. Widening validation must not silently turn the final model into an equal-weight average of old models.

The native graph selector uses recent-cutoff performance, older-cutoff stability, uncertainty and a bounded complexity preference. This is a heuristic for choosing among observed candidates. It cannot recover a graph discarded before full fidelity, nor guarantee the globally best program. The additional finalist comparison below specifically checks the production fitting lifecycle.

In the released source, `kurversc/search.py` defines configurations, balanced proposals, challengers and frontier retention. `kurversc/core.py` orchestrates their evaluation and frozen replay. The selected-configuration runner fixes the staged profile's budgets. GraphReduce's graph and node modules implement the traversal, keyed joins and generated SQL reductions. These source pointers identify the implementation of the method specified here [2], [4].

7 A symmetric production-faithful finalist check

After full rerank, S2 considers at most one smaller completed finalist. It must have a frozen plan and complete evidence on at least two cutoffs. Screening compares recent validation loss with the best completed recent loss; the candidate must be within 1%. The bounded check proceeds only for an alternate with at most half the incumbent's feature count and sufficient evidence and capacity. Otherwise it retains the incumbent.

Binary ranking and numeric regression use a common lower-is-better scale:

$$L(s) = \begin{cases} 1 - s, & s = \text{ROC-AUC}, \\ s, & s = \text{MAE}. \end{cases}$$

Here `1 - AUC` is pairwise ranking loss, counting a tied positive/negative pair as half an error. It is not classification error. At AUC 0.80, a 1% relative loss tolerance is 0.002 AUC; at AUC 0.98 it is 0.0002 AUC. The same rule therefore has comparable sensitivity on a relative loss scale rather than allowing a fixed 0.01 AUC loss near perfect ranking.

Incumbent and candidate are compared on matching origins and seeds 0, 1 and 2, using the native initial-fit/observed-validation-refit lifecycle. The native metric, uncertainty estimate and graph selector are applied independently for each seed. Replacing the incumbent requires unanimous challenger selection and, for every compared seed and fold,

$$L_{\text{candidate}} \leq 1.01 L_{\text{incumbent}} + 10^{-12}.$$

Incomplete comparisons do not promote a partially evaluated alternate. Capacity preflights and a nested stage timeout bound this extra work. This comparison is not another unrestricted graph search and does not change the downstream learner budget.

Earlier versions applied S2 only to MAE. Version 0.2.0 extends it to both supported metrics using the same comparison mechanics. Thirty regression before/after comparisons reproduce the prior MAE decisions and evidence exactly. Classification has explicit acceptance and veto coverage. Neither check establishes that the smaller graph will generalize better in every case.

8 Temporal training policies and the generic optional menu

8.1 Training support and deployment lifecycle

S3 evaluates complete temporal fitting policies against the selected graph on up to three eligible origins and seeds 0, 1 and 2. The initial incumbent uses the most recent complete training cutoff, with the declared observed-validation refit lifecycle. Alternatives include bounded pooled support, ordered regression readouts and temporal blends. The menu can consider up to twelve support cutoffs with a row floor of 1,000, subject to maturity and available history.

Pooled training applies a shared model-row budget; frame ensembles can fit that budget per member. These have different computational and statistical meanings. Three validation seeds test repeatability of selection; they are not a default three-seed inference ensemble. Deployment uses seed zero. Nor does selecting three graph finalists imply that prediction averages three graphs: one graph is selected before temporal policy and optional readout selection.

8.2 One optional menu and one incumbent

Each optional candidate challenges the same S3 incumbent, on the same phase exports and validation cases. Eligibility follows required inputs and declared capacity, independently of whether another optional candidate wins. There is no recovery cascade activated only when a preceding head keeps the incumbent.

Candidate group	Applicable inputs and globally shared action
Parent text	Eligible root categorical/text payload; bounded TF-IDF plus logistic or ridge readout, blended at 0.25 or 0.5.
Frame ensembles	Three or five initial historical frames, plus declared observed-label refit cohorts; fit separate frame members and combine predictions.
Target domain	Regression targets observed to be nonnegative or integer counts; clip negative predictions or apply the integer-domain projection.
Zero-aware	Nonnegative regression with adequate zero and positive examples; occurrence model plus positive-amount quantile readout, blended at 0.125, 0.5 or 1.
Object breadth	Declared timed root-to-object relationships; object-history features with a fitted learner, full prediction or a 0.5 blend.

There are eleven named candidate recipes before applicability filtering. The menu is fixed globally, not chosen from a table of benchmark outcomes. The following settings define bounded implementations, not an assertion that these values are optimal for every database.

The text head uses one shared text definition: at least 20 distinct values and mean nonempty length at least 32 characters. It uses at most eight root fields, truncates each to 2,048 characters, fits a training-only TF-IDF vocabulary of at most 16,384 word unigrams/bigrams, and samples at most 100,000 training examples. Classification uses logistic regression with $C = 1$; regression uses ridge regression with $\alpha = 10$. This lexical head does not require MiniLM, PyTorch or Transformers. Character length is an imperfect proxy for language; admitting a column does not prove it contains semantic text.

Target-domain projection uses eligible observed training targets, not test targets. The zero-aware candidate requires at least 20 zero and 20 positive examples and fits positive-amount quantiles at $\{0.05, 0.25, 0.5, 0.75, 0.95\}$. Its readout approximates a predictive mixture with a mass at zero [6].

Object breadth considers timed declared paths to keyed objects with at least 64 visible distinct objects, retaining the top two paths by distinct-object count. Its summaries cover object/event counts, new and singleton objects, maximum share and concentration over horizon-scaled windows. Source lookback is capped at 730 days. At most

two paths produce 42 features. The cap can truncate a long-horizon window; using more eligible paths and scaling history limits with the task are remaining generality improvements, not changes in this release. The frozen aggregate query runs serially from the first query to avoid an unsafe native DuckDB parallel-combine path without changing the query, rows or features.

8.3 Shared temporal admission rule

The optional selector requires at least two origins and three seeds. Every latest-origin seed must improve over the incumbent. Older origins supply a practical safety veto: no more than 0.005 AUC or 1% relative MAE degradation under the selector's older-origin checks. A paired, entity-cluster bootstrap uses 600 draws with $\alpha = 0.05$ and a menu-size multiplicity adjustment; the temporal gain lower bound must be positive [7]. If adequate support is unavailable, the incumbent stays selected.

The recent-only lower bound is recorded as a diagnostic and is **not** a separate positive-bound requirement. This definition is shared by optional heads. S2's graph-complexity comparison is a separate operation with the explicit relative-loss rule above, not this optional admission test.

These checks reduce unsupported promotions. They do not guarantee no future regression, account for every hypothesis examined during development, or prove transfer to an unrelated database. A global blend menu is still a finite search space; adequate fresh evaluation remains necessary.

9 Execution, portability, and production use

9.1 Explicit task context

A task supplies source tables and keys, relationships, event/availability clocks, a root entity, a label horizon, observed labels, and masked forecast requests. `TaskContext.from_tables` supports caller-owned unregistered schemas; `TaskContext.from_protocol_splits` supports already censored protocol phases. Neither requires a RelBench dataset name or a saved winning graph.

```
python -m pip install "kurversc[staged]==0.2.0" "graphreduce==1.10.18"
```

```
import kurversc

# context declares the schema, observed labels, clocks, and masked requests.
result = kurversc.run_staged_task(
    context,
    output="artifacts/task",
    task_hours=23.75,
    minimum_available_memory_gib=8,
)
predictions = result.predictions
```

Prediction artifacts preserve entity/cutoff keys and ordering. Manifests record context/source identities, selected graph and execution plan, temporal policy, optional admission and prediction fingerprints. Resume uses matching completed artifacts and retains the original absolute task deadline; it does not grant a new day for each restart. Incomplete resource-stopped tasks are not reported as successful scored tasks.

9.2 Benchmark boundary

The RelArena adapter invokes the packaged staged entry point with an explicit portable context. The harness owns outer held-out scoring. A standalone research campaign that accesses a raw RelBench cache is a different execution path and cannot establish that a harness reproduction occurred. The reported release qualification uses the registered adapter and actual core split types; the completed reproduction identifies the adapter, public packages, source boundaries and final result rows [1], [8].

The package's transitive staged execution closure includes 32 stage modules. Their execution is fixed by the packaged procedure; other repository scripts are not implicit runtime choices. The staged extra does not require a deep-learning semantic encoder.

9.3 Warehouse backends and the Kurve product

GraphReduce separates relational operations from execution, with backend support that includes production SQL environments such as Snowflake, Databricks and Redshift. SQL compilation and warehouse execution can place relational feature computation near governed source data. The frozen benchmark path uses DuckDB [4], [9]. It does not demonstrate that every staged Python-side materialization and fit runs entirely inside every warehouse, or that any SQL engine is interchangeable without backend qualification.

The Kurve product provides schema/metadata graphs, parent-node and compute-graph workflows, execution and model workflows, and integration with existing ML/MLOps stacks. Customer-controlled deployment can keep data and model operation in the customer's environment, supporting sovereign-AI requirements [10], [11]. These are production delivery options; sovereignty also depends on deployment, access controls and external-service configuration. They are distinct from benchmark rank and from an unrestricted scale claim for this search implementation.

10 Maintainer reproduction and release qualification

10.1 Release qualification (November 4, 2026)

Check	Evidence and scope
Source suite	727 passed, one skipped, on the release source.
Adapter/discovery suite	19 passed against the current registered RelArena interface.
Finalist regression parity	Thirty matched comparisons preserve prior MAE decisions and evidence.
Classification finalist checks	Explicit positive/veto cases for ranking loss, uncertainty, origins and seeds.
Installed classification run	Complete registered procedure on a fresh, unregistered 48-request synthetic schema; no hidden targets.
Installed regression run	The same execution qualification for numeric regression on a different fresh schema.
Package parity	Source and installed public-wheel Python payloads match: 80 KurveRSC and 22 GraphReduce modules.
Dependency lock	The submitted 215-package lock resolved both exact versions from PyPI.

The two synthetic runs completed in approximately 161 and 156 seconds. Their small schemas do not exercise every possible head or guarantee an S2 alternate is eligible. They establish end-to-end execution, not meaningful predictive quality or a large-database resource envelope. Qualification reused existing numerical dependencies; it is not a cold installation of every dependency on every supported platform. Detailed source checks are recorded in the qualification receipt.

10.2 Reproduced evaluation and provenance

RelArena PR #80 was merged on November 5, 2026. The maintainers ran `kurversc-2026-10-22` through its registered adapter using public KurveRSC 0.2.0 and GraphReduce 1.10.18, at seed zero, one task per CPU host. The 21 selected rows in `baseline_results/results.csv` cover seven databases, twelve binary classification tasks and nine regression tasks. All final rows contain finite test scores and no error. These are the reference measurements for this report [1].

The snapshot is RelArena `fcab8fb` (November 5, 2026), after the staged-system merge and dating of the earlier KurveRSC registration. The immutable result file, published leaderboard and resource record identify the exact comparison. The machine-readable extract retains full-precision scores, timings, source checksums and model identifiers.

10.3 Combined leaderboard

The method column credits the underlying model, architecture, pretraining-data work or official release, as applicable. RT-PluRel is credited to the RT and PluRel papers; RT-J is credited to those foundations and the RT-J

paper. The constant predictors use RelArena's documented baselines. These method citations identify intellectual provenance; every displayed score and interval still comes from the frozen RelArena reproduction, rather than from the cited papers.

The published **models plus systems** panel contains sixteen complete methods on the same 21 tasks. It excludes self-reported `_MR` reference rows and incomplete/exploratory runs. Ratings are anchored to the global constant at 1000. Intervals use the 2.5th and 97.5th percentiles of 100 task-bootstrap samples; they describe uncertainty across tasks, not variation across training seeds. A models-only board is computed separately and excludes KurveRSC as a system; its ratings must not be substituted into this combined comparison [1].

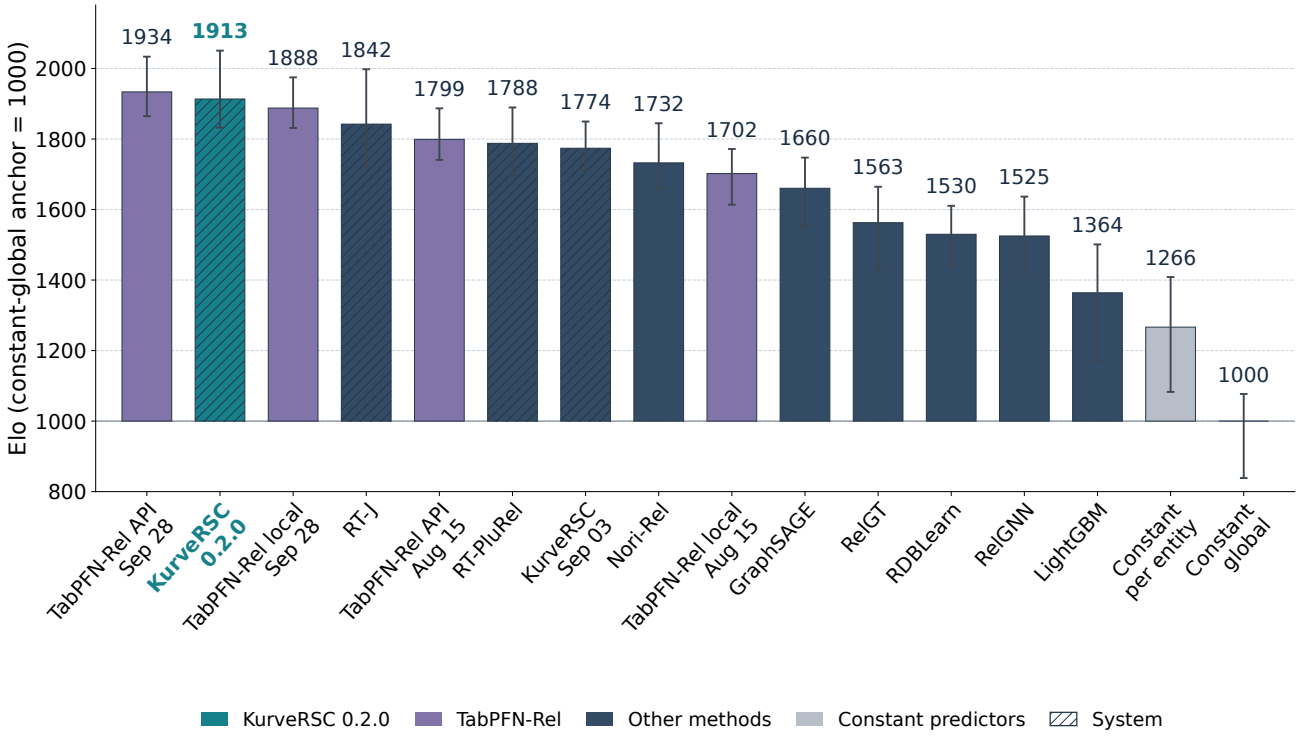


Figure 4: Combined RelArena models-and-systems leaderboard on 21 tasks. Bars show published Elo ratings above the constant-global anchor of 1000; error bars show the 2.5th-97.5th percentiles of 100 task-bootstrap samples. Hatched bars identify systems; KurveRSC 0.2.0 is highlighted. Point labels are rounded to whole Elo; the table retains published precision. All values use the same November 5, 2026 snapshot, and overlapping intervals do not establish a statistically significant ordering.

Published ratings and method references for the same combined panel:

Rank	Method	Kind	Elo	95% task-bootstrap interval
1	tabpfn-rel-client-latest [8], [12]	Model	1933.6	1864.7-2033.5
2	kurversc-latest (this work)	System	1913.4	1832.3-2050.6
3	tabpfn-rel-local-latest [8], [12]	Model	1887.7	1831.3-1974.8
4	rt-j [13], [14], [15]	System	1842.2	1721.9-1997.8
5	tabpfn-rel-client-2026-08-15 [8], [16]	Model	1799.0	1740.7-1886.9
6	rt-plurel [13], [14]	System	1787.7	1695.9-1889.4
7	kurversc-2026-09-03 [3]	System	1773.8	1709.9-1849.6
8	nori-rel [17], [18]	Model	1732.3	1660.5-1844.7
9	tabpfn-rel-local-2026-08-15 [8], [16]	Model	1702.0	1613.7-1771.7
10	graphsage [19], [20]	Model	1660.3	1550.8-1747.2
11	relgt [21]	Model	1562.7	1430.3-1664.5
12	rdblearn [22]	Model	1529.8	1437.8-1610.5
13	relgnn-es [23]	Model	1525.0	1421.2-1636.6
14	lightgbm [24]	Model	1363.9	1168.9-1501.1
15	constant-per-entity [8]	Model	1266.5	1082.8-1408.5
16	constant-global [8]	Model	1000.0	838.5-1076.9

KurveRSC's point rating is 20.2 below the latest TabPFN-Rel client and 139.6 above the earlier KurveRSC registration in this same panel. The intervals overlap substantially. A higher point rank is not proof of a significant pairwise advantage, and task-level wins are not interchangeable with Elo: Elo accounts for results against the full panel.

System and openness categories. RelArena registers a method as a system when it supplies its own tuning procedure, whereas model submissions use the framework's standardized tuning regime. KurveRSC has the highest point Elo among the registered systems in this combined panel [1].

It also has the highest point Elo among fully open-source methods in the panel. Here this means that the complete evaluated prediction procedure and required model components are available under open-source licenses and can execute locally without a proprietary prediction service. This openness cohort is our classification of the published entries, not a separate RelArena submission category or a recomputed Elo board. The evaluated MIT-licensed KurveRSC release uses local GraphReduce, DuckDB and CatBoost components [2], [4], [5], [9].

The only higher-ranked entry is the hosted TabPFN-Rel client, at 1933.6 Elo. The TabPFN-3.5 report distinguishes proprietary API implementations from its released local checkpoints [12]. The openness label describes the evaluated procedure rather than the model-family name; local TabPFN-Rel variants remain in the comparison. These category ranks describe aggregate point estimates for the stated snapshot, rather than superiority on every task or a statistically significant ordering.

10.4 Classification: ROC-AUC, higher is better

The tables compare KurveRSC 0.2.0 (`kurversc-2026-10-22`) with the latest TabPFN-Rel API and local variants (both dated September 28, 2026), RT-J, RT-PluRel, Nori-Rel and GraphSAGE. This set covers strong aggregate competitors and several relational modeling approaches; the combined leaderboard above provides their method citations. The short headers API and Local denote the two TabPFN-Rel variants. All values are selected held-out test scores from the same frozen RelArena file. **Bold marks the best score among the seven displayed methods**, determined before rounding; it does not indicate statistical significance. Appendix B includes all sixteen methods, including earlier releases and simpler baselines [1].

Dataset / task	KurveRSC	API	Local	RT-J	RT-PluRel	Nori-Rel	GraphSAGE
Amazon / item-churn	0.828712	0.828653	0.827685	**0.832556**	0.831014	0.822805	0.830527
Amazon / user-churn	0.712558	0.711698	0.706670	**0.714327**	0.713592	0.690798	0.704596
Avito / user-clicks	0.660062	0.691631	**0.698621**	0.671975	0.670345	0.636418	0.608674
Avito / user-visits	**0.672515**	0.672453	0.671587	0.667930	0.662691	0.662070	0.665758
Event / user-ignore	0.850888	0.784037	0.817646	0.821496	0.823280	**0.859148**	0.758728
Event / user-repeat	0.782152	0.760495	0.760628	**0.786598**	0.777340	0.781217	0.784626
F1 / driver-dnf	0.722017	**0.727809**	0.724794	0.588469	0.643268	0.715532	0.717235
F1 / driver-top3	**0.834154**	0.791702	0.810632	0.797757	0.750144	0.793922	0.725975
H&M / user-churn	**0.707174**	0.706021	0.705416	0.706981	0.703784	0.694094	0.698525
Stack / user-badge	0.876020	0.883443	0.884633	**0.893610**	0.893398	0.840720	0.888748
Stack / user-engagement	0.903861	0.907662	**0.907741**	0.904240	0.895384	0.896160	0.905609
Trial / study-outcome	0.760476	**0.764556**	0.758230	0.711371	0.697735	0.742175	0.686232

10.5 Regression: MAE, lower is better

MAE is in each task's native units. These values are not averaged across tasks with different target scales. Bold uses the same seven-method comparison.

Dataset / task	KurveRSC	API	Local	RT-J	RT-PluRel	Nori-Rel	GraphSAGE
Amazon / item-ltv	**42.3828**	45.2705	45.3496	42.5622	42.8036	43.7084	49.2453
Amazon / user-ltv	13.9894	13.8771	13.8858	**13.8515**	13.8798	14.1412	14.4153
Avito / ad-ctr	0.033258	**0.031258**	0.031350	0.034591	0.036435	0.031766	0.038966
Event / user-attendance	0.234934	0.241652	0.238005	0.241660	0.241207	**0.233820**	0.245018
F1 / driver-position	4.0605	**3.7574**	3.7785	3.7911	3.8535	3.8844	4.0111
H&M / item-sales	**0.033636**	0.036479	0.036539	0.037345	0.039304	0.043806	0.055153
Stack / post-votes	0.062864	**0.062733**	0.062842	0.068049	0.064124	0.065047	0.064898
Trial / site-success	0.398749	0.377615	0.404219	0.362282	0.371274	0.417738	**0.324851**
Trial / study-adverse	33.1750	34.4014	35.5059	37.4920	34.0414	**30.8832**	44.3153

KurveRSC wins **11/21** observed task comparisons against the API client: seven classification and four regression tasks. The client wins the other ten. These are pairwise comparisons, not counts of best scores across all methods. Several margins, especially item-churn and user-visits, are small point advantages; a single training seed does not establish their robustness. Relative to the earlier KurveRSC registration, fifteen scores improve and six regress. Appendix B reports both versions; the comparison is not a controlled ablation of any single feature or selection stage.

11 Cost, resources, and parallelization

11.1 Measured resources in the maintainer reproduction

The public resource record reports successful jobs on two CPU host classes: eight tasks on 96-vCPU, 739-GB hosts, and thirteen on 48-core, 190-GB hosts. The larger hosts used a 160-GiB DuckDB working-memory limit and a 100-GiB temporary-directory limit; the smaller hosts used 48 GiB and 20 GiB, respectively. One task ran per host. Host RAM is reported in GB; observed Slurm `MaxRSS` is reported in GiB [1].

Task	Host RAM, GB	Job wall time, h	Peak RSS, GiB
Amazon / item-churn	739	3.32	364
Amazon / item-ltv	739	6.48	202
Amazon / user-churn	739	4.98	262
Amazon / user-ltv	739	8.86	327
Avito / ad-ctr	190	0.71	34
Avito / user-clicks	190	1.21	51
Avito / user-visits	190	1.14	64
Event / user-attendance	190	3.35	46
Event / user-ignore	739	1.83	283
Event / user-repeat	190	1.72	44
F1 / driver-dnf	190	5.88	6
F1 / driver-position	190	4.88	4
F1 / driver-top3	190	3.93	4
H&M / item-sales	190	3.47	27
H&M / user-churn	190	1.64	21
Stack / post-votes	739	10.60	257
Stack / user-badge	739	10.38	370
Stack / user-engagement	739	4.36	219
Trial / site-success	190	1.73	89
Trial / study-adverse	190	6.36	30
Trial / study-outcome	190	2.27	23

The successful jobs take **0.71-10.60 hours** and report **4-370 GiB MaxRSS**. The separate application `time_total` values in the score CSV span 0.70-10.59 hours and sum to 88.81 task-hours. Job wall time and application time are different recorded clocks. Their sum is not the elapsed duration of a parallel multi-host campaign and does not include failed earlier attempts. Every successful reproduced task finishes below 24 hours.

Resource retries are part of the provenance: user-ignore and user-engagement ran out of memory on the 190-GB hosts with the 48-GiB DuckDB limit; post-votes reached 140 GiB there before being moved. All three completed on the 739-GB hosts. Consequently, this is a complete set of final results, **not a claim of 21 successful first attempts**. Peaks depend on engine limits and execution conditions and are observations, not minimum-memory requirements. Slurm MaxRSS is not interchangeable with the earlier author's process-tree RSS or cgroup memory peaks [1].

11.2 Execution budgets and parallel admission

The complete task budget is 23.75 hours, below RelArena's 24-hour limit. This is a deadline, not a guarantee that every schema can finish under it. Source size, join fan-out, emitted columns, historical cutoffs and optional learners all contribute to cost. A timeout or resource stop must remain visible in the results; it must not silently shorten the procedure and be called a complete reproduction.

Training-row and feature limits do not cap total RAM. SQL joins, annotations, frozen-frame caches, Python copies, CatBoost workspaces and concurrent graph workers may coexist. DuckDB working-memory settings govern spill thresholds, not the entire process tree. Several simultaneously active tasks can therefore require much more memory than the largest model matrix.

Current execution reuses matching plans and materializations rather than restarting a search after each later comparison. It separates source replay from model fitting where possible and avoids rediscovery during prediction. Such

reuse must preserve phase, source identity, cutoff, operations and schema; a cache hit with mismatched bindings is a correctness error.

Task-level and graph-configuration concurrency are distinct. Increasing both multiplies overlapping materialization and learner memory. Parallel execution should be admitted from measured process-tree or cgroup demand with an explicit host reserve. The maintainer measurements above provide current planning evidence, but concurrent tasks need additional headroom and may change spill behavior.

The published resource table identifies host classes, job wall time, Slurm MaxRSS and engine limits. It does not report every temporary-storage peak or CPU-utilization trace. Hardware-normalized speedups cannot be inferred from measurements on different hosts with different accounting methods. Further efficiency work should retain source/phase identity and disclose retries, concurrency and the resource measure used.

12 Related work and architectural claims

12.1 Relational feature construction and learned neighborhood representations

Deep Feature Synthesis and OneBM automate relational feature construction [25], [26]. KurveRSC follows the broad idea of constructing entity-grain features from related records, with supervised program selection, explicit temporal phases and frozen execution as central requirements.

RelBench and its larger v2 repository supply relational prediction tasks [20], [27]. GraphSAGE, relational deep learning, RelGNN and RelGT instead learn representations through neighborhood/message-passing mechanisms [19], [21], [23], [28]. These can retain or learn interactions that a finite set of independent reductions erases. KurveRSC does not claim representational equivalence to those models.

Deep Sets and work on graph-neural-network expressivity motivate careful choice of aggregation functions [29], [30]. They do not prove that this bounded, hand-specified feature library is universally sufficient. Equal compressed vectors are indistinguishable to any downstream learner; a stronger tabular model cannot reconstruct information absent from those vectors.

The Relational Transformer (RT) tokenizes database cells with schema metadata and attends across relational structure [13]. PluRel supplies synthetic relational databases for pretraining [14]. RT-PluRel combines that architecture and synthetic-data pretraining. RT-J builds on these foundations with continued pretraining on real relational databases from THE JOIN and context-efficient retrieval and inference [15]. In the RelArena panel, RT-PluRel and RT-J are fine-tuned systems with custom tuning; their reproduced scores should not be read as zero-shot results from the architecture papers [8].

TabPFN and subsequent tabular foundation models use pretrained inference [12], [16], [31]. TabPFN-Rel couples this learner family with relational feature construction, while RDBLearn provides an aggregation-to-tabular-inference framework [8], [22]. Nori-Rel pairs relational feature construction with the Nori foundation model; its official release description and the authors' requested software citation provide the corresponding credit [17], [18]. The entity-only LightGBM baseline uses gradient-boosted decision trees [24], while KurveRSC's benchmark learner is CatBoost [5]. KumoRFM explores a separate relational in-context architecture [32].

The TabPFN-3.5 technical report was most recently revised November 3, 2026 at this writing [12]. The benchmark comparators are the dated API/local submissions identified in the reproduction section; their reproduced scores are distinct from paper-reported results. Model papers, training-data papers and software releases are cited for their respective contributions, rather than treated as interchangeable methods.

12.2 What this approach says about flattening

KumoRFM-2 argues that task-agnostic aggregation can obscure useful relational structure, and motivates early task conditioning in relational prediction [33]. NVIDIA Kumo's explanatory documentation likewise emphasizes losses of topology, temporal structure and multi-hop interactions when constructing flat features [34]. These are substantive representation concerns. They are not a theorem that every learner consuming a matrix must be uncompetitive.

KurveRSC makes a different design choice: select a task-conditioned relational program before fitting the tabular learner. This addresses the gap between unselected summaries and supervised relational compression. It leaves open whether the available programs preserve the interactions a particular task needs. The maintainer-reproduced second-place overall result provides empirical evidence that this matrix-based learner interface can be competitive after task-conditioned relational compression. It does not show that arbitrary flattening is lossless or establish a

matched result against KumoRFM-2. That comparison would require the same tasks, inputs, temporal visibility, external pretraining disclosure and compute protocol.

12.3 Search allocation and uncertainty

Successive halving, Hyperband and BOHB study allocation of expensive evaluations [35], [36], [37]. The present stages use a related increasing-fidelity idea, but do not inherit a guarantee that 10K or 50K graph scores preserve full-source rankings. Their usefulness must be measured for this representation space and deployment lifecycle.

Paired temporal comparisons and cluster resampling account for some correlated entity observations. They do not turn reused development tasks into untouched tests. Conservative admission is a selection mechanism, not proof that a specialized transform generalizes. Future claims should emphasize fresh-schema evaluations and controlled ablations alongside aggregate benchmark scores.

13 Limitations and next steps

The most direct failure modes are representation loss, selection regret and cost. Bounded CDFs, category annotations, event windows and peer summaries do not retain every joint event or neighbor distinction. A frozen plan discovered on capped source cohorts can miss rare categories and fine temporal interactions. A stronger graph can exist in the space yet be screened out before comparable full-fidelity evidence is collected.

The next search evaluation should measure rank correlation, winner retention and selection regret across 10K, 50K and full-source fidelity on identical candidate sets. Finalist comparisons should match deployment support and refit behavior. Reporting only the final winning score does not reveal whether an improvement came from richer signal or a better selection decision.

The separation between graph selection and S4 also limits joint optimization. Optional procedures are tested only after one graph is retained. This reduces search cost, but a graph that loses under the initial fitting recipe could be stronger with an optional representation or readout. Future architectures may evaluate compatible feature generators during graph search or jointly compare a bounded set of graph finalists and downstream recipes. Such changes must be judged under explicit search and compute budgets.

Generalization also depends on source metadata quality, historical data availability, eligibility limits and validation support. Top-two object paths, a fixed 730-day object-history cap, and heuristic text detection are disclosed restrictions. Enumerating more valid alternatives may improve generality, but can increase search cost and multiple-comparison burden. Changes should be qualified with the same public procedure on held-out databases, without benchmark-name switches or test-score campaign gates.

Future work will investigate pretrained configuration models that reduce task-time relational program search, potentially combined with pretrained downstream learners. Evaluation will measure predictive quality and computational cost on databases excluded from pretraining. This direction is outside KurveRSC 0.2.0 and the results reported here.

Acknowledgments

We thank Rishabh Ranjan for reviewing the manuscript and providing helpful feedback on related work and citations.

We also thank Adrian Hayler for his constructive methodological feedback, code review, and collaboration on the KurveRSC integration, and the Prior Labs team and RelArena maintainers for developing the benchmark and independently reproducing the reported results.

References

References are numbered in order of first citation and print complete author lists. The bibliography covers the implementation, protocol, relational feature construction, learned representations, search allocation and statistical methods. Compact source labels remain linked to the complete original URLs. Product/deployment references describe capabilities rather than benchmark results. Publication metadata and access dates are recorded in the individual reference entries.

[1] Prior Labs, "RelArena: Benchmark Software and Reproduced Results, November 5, 2026 Snapshot." 2026. Accessed: Nov. 05, 2026. [Online]. Available: GitHub source

[2] Wes Madrigal, *KurveRSC*. (Nov. 04, 2026). Accessed: Nov. 06, 2026. [Online]. Available: GitHub source

[3] Wes Madrigal, "KurveRSC: Validation-Guided Relational Signal Compression with a Downstream Learner in the Loop," Kurve AI, 2026. Accessed: Oct. 26, 2026. [Online]. Available: GitHub source

[4] Wes Madrigal, "GraphReduce." Accessed: Nov. 05, 2026. [Online]. Available: GitHub source

[5] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin, "CatBoost: Unbiased Boosting with Categorical Features," in *Advances in neural information processing systems*, 2018. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1706.09516

[6] Roger Koenker and Gilbert Bassett Jr., "Regression Quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978, doi: 10.2307/1913643.

[7] Bradley Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.

[8] Adrian Hayler, Klemens Flöge, Alan Arazi, Rishabh Ranjan, Jure Leskovec, Felix Birkel, Brendan Roof, Anurag Garg, Kristina Collins, Lydia Sidhoum, Jonas Kübler, Siyuan Guo, Oscar Key, Jan Hendrik Metzen, Rylee Grace, David Salinas, Arthur Cahu, Simon Bing, Benjamin Jäger, Tuana Çelik, Mihir Manium, Vitor Monteiro, Jake Robertson, Jerry Chen, Eliott Kalfon, Tomás Pereda, Lilly Wehrhahn, Dominik Safaric, Tobias Schroeder, Georg Grab, Diana Kriuchkova, Clara Cornu, Philipp Singer, Nick Erickson, Vahid Balazadeh, Marie Salmon, Simone Alessi, Kürşat Kaya, Philipp Jund, Léo Grinsztajn, Yann LeCun, Bernhard Schölkopf, Madelon Hulsebos, Lennart Purucker, Sauraj Gambhir, Frank Hutter, and Noah Hollmann, "Advancing Open and Reproducible Relational Learning: RelArena-alpha, TabPFN-Rel and RPI." 2026. Accessed: Nov. 06, 2026. [Online]. Available: arXiv:2608.16319v2

[9] Mark Raasveldt and Hannes Muhleisen, "DuckDB: An Embeddable Analytical Database," in *Proceedings of the 2019 international conference on management of data*, 2019. doi: 10.1145/3299869.3320212.

[10] Kurve AI, "Kurve Product and Deployment." Accessed: Nov. 05, 2026. [Online]. Available: Kurve AI

[11] Kurve AI, "Kurve Product Documentation." Accessed: Nov. 05, 2026. [Online]. Available: Kurve documentation

[12] Benjamin Jäger, Nick Erickson, Léo Grinsztajn, Felix Birkel, Klemens Flöge, Oscar Key, Kürşat Kaya, Jonas Kübler, Adèle Frankel, Tobias Schröder, Anurag Garg, Jan Hendrik Metzen, David Salinas, Simon Bing, Kristina Collins, Tuana Çelik, Vahid Balazadeh, Lydia Sidhoum, Tomás Pereda, Brendan Roof, Andrej Tschalzev, Siyuan Guo, Philipp Singer, Lennart Purucker, Jake Robertson, Marie Salmon, Philipp Jund, Jerry Chen, Diana Kriuchkova, Arthur Cahu, Eliott Kalfon, Adrian Hayler, Georg Grab, Vitor Monteiro, Lilly Wehrhahn, Dominik Safaric, Clara Cornu, Alan Arazi, Rylee Grace, Simone Alessi, Mihir Manium, Bernhard Schölkopf, Yann LeCun, Madelon Hulsebos, Sauraj Gambhir, Noah Hollmann, and Frank Hutter, "TabPFN-3.5: Technical Report." 2026. Accessed: Nov. 06, 2026. [Online]. Available: arXiv:2609.17895

[13] Rishabh Ranjan, Valter Hudovernik, Mark Znidar, Charilaos Kanatsoulis, Roshan Upendra, Mahmoud Mohammadi, Joe Meyer, Tom Palczewski, Carlos Guestrin, and Jure Leskovec, "Relational transformer: Toward zero-shot foundation models for relational data," in *The fourteenth international conference on learning representations*, 2026. Accessed: Nov. 05, 2026. [Online]. Available: arXiv:2510.06377

[14] Vignesh Kothapalli, Rishabh Ranjan, Valter Hudovernik, Vijay Prakash Dwivedi, Johannes Hoffart, Carlos Guestrin, and Jure Leskovec, "PluRel: Synthetic data unlocks scaling laws for relational foundation models," in *Forty-third international conference on machine learning*, 2026. Accessed: Nov. 05, 2026. [Online]. Available: Publisher

[15] Rishabh Ranjan, Vignesh Kothapalli, Harshvardhan Agarwal, Charilaos Kanatsoulis, Roshan Upendra, Tom Palczewski, Carlos Guestrin, and Jure Leskovec, "RT-J: Large-scale pretraining of relational transformers for context-efficient predictions," in *The fortieth annual conference on neural information processing systems*, 2026. Accessed: Nov. 05, 2026. [Online]. Available: arXiv:2610.17867

[16] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Mihir Manium, Shi Bin Hoo, Magnus Bühler, Anurag Garg, Dominik Safaric, Jake Robertson, Benjamin Jäger, Simone Alessi, Adrian Hayler, Vladyslav Moroshan, Lennart Purucker, Philipp Singer, Alan Arazi, Julien Siems, Jan Hendrik Metzen, Georg Grab, Nick Erickson, Siyuan Guo, Eliott Kalfon, Simon Bing, David Salinas, Clara Cornu, Lilly Charlotte Wehrhahn, Diana Kriuchkova, Kursat Kaya, Lydia Sidhoum, Marie Salmon, Jerry Chen, Madelon Hulsebos, Yann LeCun, Samuel Müller, Bernhard Schölkopf, Sauraj Gambhir, Noah Hollmann, and Frank Hutter, "TabPFN-3: Technical Report." 2026. Accessed: Nov. 06, 2026. [Online]. Available: arXiv:2605.13986v2

[17] Synthefy, Po-han Li, Aditya Narayanan, Sai Shankar Narasimhan, Raghav Mallampalli, Aahan Agrawal, Bekzat Ajan, Raimi Shah, and Shubhankar Agarwal, *Synthefy Nori: Tabular Foundation Model for Regression.* (Jun. 2026). Zenodo. doi: 10.5281/zenodo.20710462.

[18] Synthefy Team, "Introducing Nori-Rel: Predictions for Your Relational Databases." Accessed: Nov. 05, 2026. [Online]. Available: www.synthefy.com

[19] William L. Hamilton, Rex Ying, and Jure Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in neural information processing systems*, 2017. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1706.02216

[20] Joshua Robinson, Rishabh Ranjan, Weihua Hu, Kexin Huang, Jiaqi Han, Alejandro Dobles, Matthias Fey, Jan Eric Lenssen, Yiwen Yuan, Zecheng Zhang, Xinwei He, and Jure Leskovec, "RelBench: A Benchmark for Deep Learning on Relational Databases," in *Advances in neural information processing systems, datasets and benchmarks*, 2024. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2407.20060

[21] Vijay Prakash Dwivedi, Sri Jaladi, Yangyi Shen, Federico Lopez, Charilaos I. Kanatsoulis, Rishi Puri, Matthias Fey, and Jure Leskovec, "Relational Graph Transformer," in *International conference on learning representations*, 2026. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2505.10960v2

[22] Yanlin Zhang, Linjie Xu, Quan Gan, David Wipf, and Minjie Wang, "RDBLearn: Simple In-Context Prediction Over Relational Databases." 2026. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2602.18495

[23] Tianlang Chen, Charilaos Kanatsoulis, and Jure Leskovec, "RelGNN: Composite Message Passing for Relational Deep Learning," in *Proceedings of the international conference on machine learning*, 2025. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2502.06784

[24] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in neural information processing systems*, 2017. Accessed: Nov. 05, 2026. [Online]. Available: Publisher

[25] James Max Kanter and Kalyan Veeramachaneni, "Deep Feature Synthesis: Towards Automating Data Science Endeavors," in *IEEE international conference on data science and advanced analytics*, 2015. doi: 10.1109/DSAA.2015.7344858.

[26] Hoang Thanh Lam, Johann-Michael Thiebaut, Mathieu Sinn, Bei Chen, Tiep Mai, and Oznur Alkan, "One Button Machine for Automating Feature Engineering in Relational Databases." 2017. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1706.00327

[27] Justin Gu, Rishabh Ranjan, Charilaos Kanatsoulis, Haiming Tang, Martin Jurkovic, Valter Hudovernik, Mark Znidar, Pranshu Chaturvedi, Parth Shroff, Fengyu Li, and Jure Leskovec, "RelBench v2: A Large-Scale Benchmark and Repository for Relational Data." 2026. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2602.12606v2

[28] Matthias Fey, Weihua Hu, Kexin Huang, Jan Eric Lenssen, Rishabh Ranjan, Joshua Robinson, Rex Ying, Jiaxuan You, and Jure Leskovec, "Position: Relational Deep Learning - Graph Representation Learning on Relational Databases," in *Proceedings of the 41st international conference on machine learning*, 2024, pp. 13592–13607. Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[29] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander J. Smola, "Deep Sets," in *Advances in neural information processing systems*, 2017. Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[30] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka, "How Powerful are Graph Neural Networks?" in *International conference on learning representations*, 2019. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1810.00826

[31] Noah Hollmann, Samuel Muller, Lennart Purucker, Arjun Krishnakumar, Max Korfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter, "Accurate Predictions on Small Data with a Tabular Foundation Model," *Nature*, vol. 637, pp. 319–326, 2025, doi: 10.1038/s41586-024-08328-6.

[32] Matthias Fey, Vid Kocijan, Federico Lopez, Jan Eric Lenssen, and Jure Leskovec, "KumoRFM: A Foundation Model for In-Context Learning on Relational Data," Kumo.AI, 2025. Accessed: Oct. 26, 2026. [Online]. Available: Archived report

[33] Valter Hudovernik, Federico Lopez, Vid Kocijan, Akihiro Nitta, Jan Eric Lenssen, Jure Leskovec, and Matthias Fey, "KumoRFM-2: Scaling Foundation Models for Relational Learning." 2026. Accessed: Nov. 05, 2026. [Online]. Available: arXiv:2604.12596v1

[34] NVIDIA Kumo, "Relational Deep Learning." Accessed: Nov. 05, 2026. [Online]. Available: NVIDIA Kumo documentation

[35] Kevin Jamieson and Ameet Talwalkar, "Non-stochastic Best Arm Identification and Hyperparameter Optimization," in *Proceedings of the 19th international conference on artificial intelligence and statistics*, 2016, pp. 240–248. Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[36] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018, Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[37] Stefan Falkner, Aaron Klein, and Frank Hutter, "BOHB: Robust and Efficient Hyperparameter Optimization at Scale," in *Proceedings of the 35th international conference on machine learning*, 2018, pp. 1437–1446. Accessed: Oct. 26, 2026. [Online]. Available: Publisher

14 Appendix A: release and implementation provenance

The evaluated shared procedure is identified internally as `kurversc-generic-staged-v1`. Its public dependency recipe is:

```
kurversc[staged]==0.2.0
graphreduce==1.10.18
```

Both were published on November 4, 2026. For source audit, the release snapshots are KurveRSC `cf6599bd7de21f32b299c3cdce75f611ad050a06` and GraphReduce `21aff2e9ad83d69d7fc468d01e3d56a0dc6ca976`. The latter is the code published as GraphReduce 1.10.18, not an unpublished local patch. Documentation revisions do not alter these released Python payloads.

The public artifact checksums are recorded below. They identify release files, not additional installation constraints.

```
kurversc-0.2.0-py3-none-any.whl SHA-256
0d5eae0f9381d0948475352bcae97b85e5fc5dce98ea4cfb1f7d50bceb635ea0

kurversc-0.2.0.tar.gz SHA-256
6093ba24d9d4fb31252980d816032c1141dab6ea9c50713e797428d458f5fc20

graphreduce-1.10.18-py3-none-any.whl SHA-256
5d99b358e4ce5f85c3d16fb8a2db072d9c18da6abbd03c4e194aff822410bd81

graphreduce-1.10.18.tar.gz SHA-256
20e7ac6335c6231c836e83339d69c98c895f30800fb83d5f7484fbb57960afc1
```

Implementation area	Source entry points
Public staged execution	`kurversc/staged.py`, `scripts/run_portable_staged.py`
Fixed generic contract	`scripts/run_admissible_staged.py`
Task/source boundaries	`kurversc/task_context.py`, `kurversc/task_source.py`
Search and frozen plans	`kurversc/core.py`, `scripts/run_kurversc_selected_relarena.py`
Finalist symmetry	`scripts/generic_finalist_helpers.py`, `scripts/benchmark_production_finalists_v2.py`
Temporal selection	`kurversc/temporal_policy.py`
Shared optional menu	`scripts/benchmark_generic_readouts.py`, `kurversc/signal_challengers.py`
Object-history boundary	`scripts/phase_frozen_object_breadth.py`
Packaged stage closure	`staged_build.py`, `kurversc/staged_runtime.py`

The current release specification, review/change record and qualification receipt provide companion evidence. Reproduction uses the declared packages, source boundaries and fixed procedure.

15 Appendix B: complete per-task comparison

All sixteen methods in the combined leaderboard are included below. Panels A and B split columns for readability and follow the leaderboard order. **Bold marks the best full-precision score across both panels together** for each task. Consequently, a panel may have no bold entry on a particular row. All scores use the selected seed-zero rows from the same frozen result file; method citations are given in Section 10.3 [1].

API-09 and Local-09 are the September 28, 2026 TabPFN-Rel releases; API-08 and Local-08 are the August 15, 2026 releases. Kurve-02 is KurveRSC 0.2.0 (`kurversc-2026-10-22`); Kurve-01 is the earlier `kurversc-2026-09-03` registration. RelGNN denotes `relgnn-es`; Entity and Global denote the per-entity and global constant predictors. Scores are point estimates; bold does not establish statistical significance.

15.1 Classification: ROC-AUC, higher is better

Panel A: first eight methods in the combined leaderboard.

Dataset / task	API-09	Kurve-02	Local-09	RT-J	API-08	RT-PluRel	Kurve-01	Nori-Rel
Amazon / item-churn	0.828653	0.828712	0.827685	**0.832556**	0.827996	0.831014	0.827256	0.822805
Amazon / user-churn	0.711698	0.712558	0.706670	**0.714327**	0.708649	0.713592	0.710347	0.690798
Avito / user-clicks	0.691631	0.660062	**0.698621**	0.671975	0.675191	0.670345	0.658406	0.636418
Avito / user-visits	0.672453	0.672515	0.671587	0.667930	0.668026	0.662691	**0.674721**	0.662070
Event / user-ignore	0.784037	0.850888	0.817646	0.821496	**0.878659**	0.823280	0.832628	0.859148
Event / user-repeat	0.760495	0.782152	0.760628	**0.786598**	0.759291	0.777340	0.779679	0.781217
F1 / driver-dnf	0.727809	0.722017	0.724794	0.588469	0.732172	0.643268	**0.746484**	0.715532
F1 / driver-top3	0.791702	**0.834154**	0.810632	0.797757	0.771426	0.750144	0.664957	0.793922
H&M / user-churn	0.706021	**0.707174**	0.705416	0.706981	0.705215	0.703784	0.696916	0.694094
Stack / user-badge	0.883443	0.876020	0.884633	**0.893610**	0.880386	0.893398	0.877626	0.840720
Stack / user-engagement	0.907662	0.903861	**0.907741**	0.904240	0.905994	0.895384	0.903027	0.896160
Trial / study-outcome	0.764556	0.760476	0.758230	0.711371	**0.764702**	0.697735	0.712131	0.742175

Panel B: remaining eight methods.

Dataset / task	Local-08	GraphSAGE	RelGT	RDBLearn	RelGNN	LightGBM	Entity	Global
Amazon / item-churn	0.827857	0.830527	0.823835	0.819538	0.785579	0.662211	0.728875	0.500000
Amazon / user-churn	0.702403	0.704596	0.701924	0.684375	0.694281	0.517056	0.634205	0.500000
Avito / user-clicks	0.614522	0.608674	0.644353	0.678769	0.667571	0.564163	0.504143	0.500000
Avito / user-visits	0.668811	0.665758	0.662142	0.659624	0.648731	0.529282	0.602703	0.500000
Event / user-ignore	0.701376	0.758728	0.781507	0.664351	0.805393	0.777181	0.839930	0.500000
Event / user-repeat	0.769251	0.784626	0.734358	0.744084	0.754612	0.748295	0.751805	0.500000
F1 / driver-dnf	0.714468	0.717235	0.711667	0.714551	0.726106	0.730298	0.699258	0.500000
F1 / driver-top3	0.792916	0.725975	0.810841	0.780081	0.758864	0.738889	0.556530	0.500000
H&M / user-churn	0.705690	0.698525	0.689531	0.698352	0.682025	0.590081	0.647972	0.500000
Stack / user-badge	0.863470	0.888748	0.574286	0.771147	0.620584	0.537995	0.788956	0.500000
Stack / user-engagement	0.905834	0.905609	0.906731	0.858670	0.905054	0.811836	0.826717	0.500000
Trial / study-outcome	0.730607	0.686232	0.668495	0.721205	0.657435	0.715018	0.500000	0.500000

15.2 Regression: MAE, lower is better

MAE retains each task's native units. Panel abbreviations and cross-panel bolding are the same as above.

Panel A: first eight methods in the combined leaderboard.

Dataset / task	API-09	Kurve-02	Local-09	RT-J	API-08	RT-PluRel	Kurve-01	Nori-Rel
Amazon / item-ltv	45.2705	42.3828	45.3496	42.5622	46.7682	42.8036	**42.2731**	43.7084
Amazon / user-ltv	13.8771	13.9894	13.8858	**13.8515**	14.3582	13.8798	14.0205	14.1412
Avito / ad-ctr	0.031258	0.033258	0.031350	0.034591	**0.031080**	0.036435	0.033440	0.031766
Event / user-attendance	0.241652	0.234934	0.238005	0.241660	0.243944	0.241207	0.258668	**0.233820**
F1 / driver-position	**3.7574**	4.0605	3.7785	3.7911	3.7692	3.8535	3.8632	3.8844
H&M / item-sales	0.036479	0.033636	0.036539	0.037345	0.060507	0.039304	**0.032168**	0.043806
Stack / post-votes	**0.062733**	0.062864	0.062842	0.068049	0.067882	0.064124	0.063243	0.065047
Trial / site-success	0.377615	0.398749	0.404219	0.362282	0.412624	0.371274	0.403842	0.417738
Trial / study-adverse	34.4014	33.1750	35.5059	37.4920	39.7537	34.0414	41.2702	**30.8832**

Panel B: remaining eight methods.

Dataset / task	Local-08	GraphSAGE	RelGT	RDBLearn	RelGNN	LightGBM	Entity	Global
Amazon / item-ltv	47.7683	49.2453	48.6734	48.9975	52.4581	55.7508	65.3514	64.2336
Amazon / user-ltv	14.4009	14.4153	14.3521	14.5775	14.5752	16.7847	17.4235	16.7830
Avito / ad-ctr	0.031379	0.038966	0.036499	0.034103	0.042646	0.041250	0.041201	0.043067
Event / user-attendance	0.239383	0.245018	0.261493	0.242215	0.243858	0.262798	0.269152	0.263534
F1 / driver-position	3.7617	4.0111	4.7655	3.8886	4.2659	4.1058	4.1035	4.3991
H&M / item-sales	0.061362	0.055153	0.053168	0.067132	0.056493	0.075295	0.078033	0.076087
Stack / post-votes	0.067957	0.064898	0.067904	0.067719	0.067904	0.066099	0.069374	0.067904
Trial / site-success	0.385751	**0.324851**	0.370110	0.485833	0.340493	0.437506	0.441148	0.462222
Trial / study-adverse	42.5917	44.3153	44.1259	44.0267	46.2701	44.5735	57.5332	57.5332